Object-Oriented Programming (OOP) in Python

1. Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects**, which are real-world entities that contain: - **Data (attributes / properties)** - **Behavior (methods / functions)**

Python supports OOP and allows developers to build reusable, scalable, and maintainable applications.

Why OOP?

- Models real-world problems easily
- Improves code reusability
- Makes code easier to maintain and modify
- Supports large-scale application development

---

2. Class and Object

Class

A **class** is a blueprint or template used to create objects.

Syntax:

```
class ClassName:
    # attributes and methods
    pass
```

Object

An **object** is an instance of a class.

Example:

```
class Student:
    def study(self):
        print("Student is studying")

s1 = Student()
s1.study()
```

---

3. Attributes and Methods

Attributes

Variables that belong to a class or object.

Methods

Functions defined inside a class.

Example:

```
class Person:
    name = "Rahul"    # class attribute

    def speak(self):
        print("Hello")
```

---

4. The `__init__()` Constructor

The `__init__()` method is a special method that is automatically executed when an object is created.

Example:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print(self.name, self.salary)
```

---

5. Instance Variables vs Class Variables

Instance Variables

- Specific to each object

- Defined using `self`

Class Variables

- Shared among all objects

Example:

```
class Company:
    company_name = "ABC Pvt Ltd"  # class variable

    def __init__(self, emp_name):
        self.emp_name = emp_name    # instance variable
```

🎯 Four Pillars of Object-Oriented Programming

Encapsulation

Encapsulation means bundling data and methods together and restricting direct access.

Access Modifiers in Python (by convention)

- `public` → `name`
- `protected` → `_name`
- `private` → `__name`

Example:

```
class Bank:
    def __init__(self, balance):
        self.__balance = balance
```

```
    def get_balance(self):
        return self.__balance
```

Inheritance

Inheritance allows one class (child) to reuse properties and methods of another class (parent).

💡 Real-Life Analogy

A child inherits features like surname or property from parents.

Types of Inheritance

- Single
- Multilevel
- Multiple

Example (Single Inheritance):

```
class Vehicle:
    def move(self):

        print("Vehicle is moving")


class Bike(Vehicle):
    def ride(self):

            print("Bike is riding")

b = Bike()
b.move()
b.ride()
```

✅ Advantages:

• Code reuse
• Faster development
• Easy maintenance

Polymorphism

Polymorphism means **“many forms.”**

In programming, polymorphism allows **the same function name, method name, or operator** to behave **differently depending on the object or data type**.

👉 *One interface, multiple behaviors.*

Real-life example

- The word **“play”**
  - Play a **song**
  - Play a **game**
  - Play a **role**

Same word, different meanings → **polymorphism**

Method Overriding

When a **child class provides its own implementation** of a method already defined in the parent class.

Rules:

- Method name must be the same
- Parameters must be the same
- Happens in **inheritance**

```
class Animal:
    def sound(self):
        print("Animal makes a sound")
```

```
class Dog(Animal):
    def sound(self):
        print("Dog barks")


class Cat(Animal):
    def sound(self):
        print("Cat meows")


a = Animal()
d = Dog()
c = Cat()


a.sound()
d.sound()
c.sound()
```

Output:

```
Animal makes a sound
Dog barks
Cat meows
```

Method Overloading

Python does not support traditional method overloading, but it can be achieved using default arguments, *args, and conditional logic*.

```
class Abc:
    def xyz(self, a, b=None, c=None):
        if b is None and c is None:
            print("One argument:", a)
        elif c is None:
            print("Two arguments:", a + b)
        else:
            print("Three arguments:", a + b + c)

obj = Abc()
obj.xyz(10)
obj.xyz(10, 20)
obj.xyz(10, 20, 30)
```

---

10. Abstraction

Abstraction in OOPS is achieved by defining abstract classes and methods that specify behavior without providing implementation.

Python uses **ABC (Abstract Base Class)** for abstraction.

Example:

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
```

```
    def speed(self):
        pass

class Bike(Vehicle):
    def speed(self):
        print("60 km/h")
```

GENERATORS

1. What is a Generator?

A **generator** is a function that **produces values one at a time** instead of returning all values at once.

👉 It uses the keyword **`yield`** instead of `return`.

2. Why Generators are Needed?

- Save memory
- Improve performance
- Useful for large data processing
- Values are generated **on demand**

3. Difference: `return` vs `yield`

`return`	`yield`
Ends function execution	Pauses function execution
Returns one value	Returns multiple values one by one
Function memory cleared	State is remembered

4. Simple Generator Example

```
def my_generator():
    yield 1
    yield 2
    yield 3

g = my_generator()

print(next(g))
print(next(g))
print(next(g))
```

5. Generator Using Loop

```
def count_upto(n):
    for i in range(1, n + 1):
        yield i

for num in count_upto(5):
    print(num)
```

DECORATORS

What is a Decorator?

A **decorator** is a function that **modifies the behavior of another function** without changing its code.

👉 Functions are treated as **objects** in Python.

Why Decorators are Needed?

- Code reusability
- Separation of concerns
- Clean & readable code
- Used in logging, authentication, timing

Basic Function Without Decorator

```
def greet():
    print("Hello")

greet()
```

Function With Decorator

Decorator Function

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
```

```
        func()
        print("After function call")
    return wrapper
```

Apply Decorator

```
@my_decorator
def greet():
    print("Hello")

greet()
```

Output:

```
Before function call
Hello
After function call
```

Regular Expressions

1. What is a Regular Expression?

A **Regular Expression (Regex)** is a **pattern** used to **search, match, or manipulate text**.

👉 It helps us answer questions like:

- Does this text follow a specific format?
- Can I find a word inside a large text?
- Can I extract emails, phone numbers, etc.?

2. Why Use Regular Expressions?

- Validate input (email, phone number, PIN)
- Search text efficiently
- Extract specific data
- Replace patterns in text

3. Regex in Python

Python provides the **`re` module** for working with regular expressions.

```
import re
```

4. Basic Pattern Matching Functions

Function	Purpose

`re.match()`	Matches pattern at the **start**
`re.search()`	Finds first match anywhere
`re.findall()`	Finds **all matches**
`re.finditer()`	Returns iterator of matches
`re.sub()`	Replaces matched text

5 `re.match()` – Match at Beginning

```
import re

text = "Python is powerful"

result = re.match("Python", text)

if result:
    print("Match found")
else:
    print("No match")
```

✔Matches only at start

6. `re.search()` – Search Anywhere

```
result = re.search("powerful", text)
print(result.group())
```

✔ Searches entire string
✔ Returns first match

7. `re.findall()` – Find All Matches

```
text = "cat bat rat mat"

result = re.findall("at", text)
print(result)
```

Output:

```
['at', 'at', 'at', 'at']
```

SQLite in Python

SQLite is a **lightweight, serverless, self-contained** database engine that is widely used for embedding databases in applications. It is a great option for small to medium-sized applications or for development purposes where the overhead of a server-based database (e.g., MySQL, PostgreSQL) is not needed.

Python has built-in support for SQLite via the **`sqlite3`** module, which provides an easy-to-use interface to interact with SQLite databases.

1. Setting Up SQLite in Python

To interact with an SQLite database in Python, you need to use the `sqlite3` module, which is part of Python's standard library.

```
import sqlite3
```

2. Creating and Connecting to a Database

When you connect to a database using `sqlite3.connect()`, SQLite will create the database file if it does not already exist.

```
# Connect to a database (it creates the database file if it
doesn't exist)
conn = sqlite3.connect('mydatabase.db')

# Create a cursor object to interact with the database
cursor = conn.cursor()
```

3. Creating a Table

Once connected, you can execute SQL queries through the cursor object. Let's create a simple table called `users` with some columns.

```
# Create a table (if it doesn't already exist)
cursor.execute('''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER,
    email TEXT
)
''')

# Commit the transaction to save the changes
conn.commit()
```

- **`CREATE TABLE IF NOT EXISTS`**: This SQL statement creates a table if it doesn't already exist.
- **`PRIMARY KEY`**: The `id` column is set as the primary key, meaning it must have unique values.
- **`NOT NULL`**: The `name` field cannot be empty.

4. Inserting Data

To insert data into the table, you can use the `INSERT INTO` SQL statement. Here's how to insert a record into the `users` table:

```
# Inserting a single record
cursor.execute('''
INSERT INTO users (name, age, email) VALUES (?, ?, ?)
''', ('John Doe', 30, 'john.doe@example.com'))

conn.commit()

# You can also insert multiple records
cursor.execute('''
INSERT INTO users (name, age, email) VALUES (?, ?, ?)
''', [
    ('Alice', 25, 'alice@example.com'),
    ('Bob', 35, 'bob@example.com')
])
conn.commit()
```

5. Querying Data (SELECT)

To retrieve data from the table, you use the `SELECT` statement. You can fetch all or specific records based on conditions.

```
# Select all records
cursor.execute('SELECT * FROM users')

# Fetch all rows of the query result
rows = cursor.fetchall()

# Iterate over the result
for row in rows:
    print(row)  # Each row is a tuple
```

If you want to select specific columns or apply conditions (e.g., where the age is greater than 30), you can modify the query like this:

```
# Select specific columns and apply conditions
cursor.execute('SELECT name, email FROM users WHERE age >
30')

# Fetch the result
rows = cursor.fetchall()

for row in rows:
    print(row)
```

- **`fetchall()`**: Returns all the rows from the result set.
- **`fetchone()`**: Returns the next row from the result set, or `None` if no more rows are available.

6. Updating Data

To update data in the table, you use the `UPDATE` SQL statement. This is often done with a condition (`WHERE`) to specify which rows should be updated.

```
# Update a record (change the age of John Doe)
cursor.execute('''
UPDATE users SET age = ? WHERE name = ?
''', (31, 'John Doe'))
conn.commit()
```

- **`UPDATE`**: Modifies existing records.
- **`SET`**: Specifies the new value for the columns.
- **`WHERE`**: Filters which rows to update.

7. Deleting Data

To delete records from a table, use the `DELETE` SQL statement with a condition.

```
# Delete a record (where name is 'Bob')
cursor.execute('''
DELETE FROM users WHERE name = ?
''', ('Bob',))

conn.commit()
```

- **`DELETE FROM`**: Deletes records from the table based on a condition (`WHERE`).

If you omit the `WHERE` clause, all records in the table will be deleted, so use it carefully.